

Karger's Randomised Minimum Cut Algorithm

Implementation and Empirical Study

Nasr Mohammed - 24830554

(a) What I built

For this project, I implemented Karger's randomised minimum cut algorithm in C++. The algorithm finds a cut in an undirected multigraph by repeatedly selecting an edge uniformly at random and contracting its endpoints until only two supernodes remain. The edges remaining between the final two supernodes form the cut returned by that run.

The graph is represented using an edge list. I chose this representation because Karger's algorithm must sample uniformly from the currently surviving edges. It also naturally preserves parallel edges, which are important because every parallel copy counts as a separate possible random choice.

To represent contractions, I used a disjoint-set union (DSU) data structure with path compression and union by rank. Rather than physically rewriting the graph whenever two vertices are contracted, the DSU records which original vertices currently belong to the same supernode.

```
class DSU
{
private:
    std::vector<int> parent;
    std::vector<int> rank;

public:
    DSU(int n) : parent(n), rank(n, 0)
    {
        for (int i = 0; i < n; ++i)
        {
            parent[i] = i;
        }
    }

    int find(int x)
    {
        if (parent[x] != x)
        {
            parent[x] = find(parent[x]);
        }

        return parent[x];
    }
}
```

During each contraction step, the implementation scans the original edge list and keeps only edges whose endpoints belong to different DSU components. This removes self-loops created by earlier contractions while preserving parallel edges between distinct supernodes. A uniformly random surviving edge is then selected and its two components are merged. The process stops when exactly two components remain, and the crossing edges between them define the returned cut.

For reproducibility, the experiments use a `std::mt19937` random-number generator with a fixed seed. In addition to the randomised algorithm, I implemented an exact brute-force minimum-cut baseline. The exact method enumerates possible vertex partitions, fixes vertex 0 on one side to avoid duplicated complements, counts crossing edges, and returns the smallest cut found. This baseline is only practical for small graphs because the number of partitions grows exponentially, but it provides ground truth for the experiments.

I also implemented generators for three graph families: cycle graphs, complete graphs, and two-cluster graphs consisting of two dense components joined by two bridge edges. The experiment harness runs Karger's algorithm 5,000 times on each graph, compares each returned cut with the exact minimum cut, records success rate and runtime in CSV format, and generates plots using Python, pandas, and Matplotlib.

(b) Track A - Empirical Study

Research questions

The main research question was: How does the empirical probability that Karger's algorithm finds a minimum cut compare with its theoretical guarantee, and how much does graph structure affect this probability?

A secondary question was how runtime changes with graph size and density, using the exact brute-force implementation as a baseline for determining the true minimum cut.

Hypotheses

H1: The observed success rate will usually be substantially higher than Karger's theoretical lower bound because the bound is a worst-case guarantee.

H2: Graph structure will have a strong effect on success probability, so graphs with the same number of vertices can behave very differently.

H3: Denser graphs will take longer in this implementation because each contraction step scans the full edge list to rebuild the list of surviving edges.

Experimental setup

Graphs with $n = 4, 6, 8, 10$, and 12 vertices were tested. For each graph, Karger's algorithm was executed 5,000 times. A trial counted as successful when the cut returned by Karger matched the exact minimum-cut value produced by the brute-force baseline.

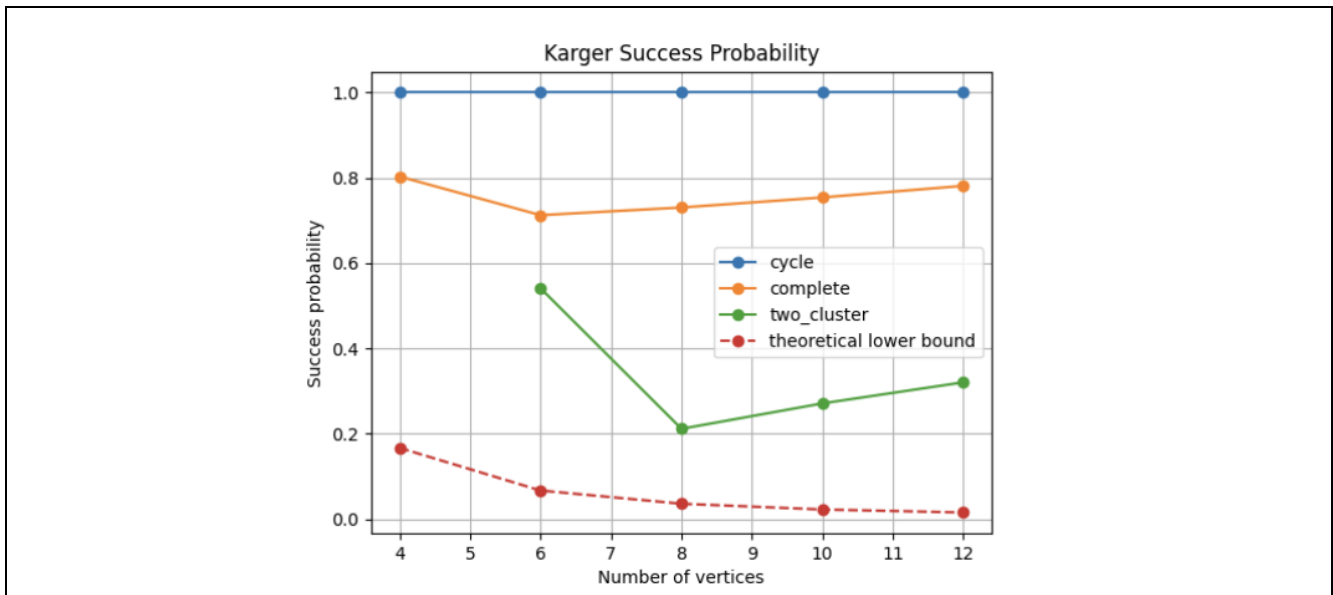
The three graph families were cycle graphs, complete graphs, and two-cluster graphs. The two-cluster family consists of two complete subgraphs connected by two bridge edges, creating an obvious sparse bottleneck. A fixed random seed was used so that the experiment is reproducible.

The theoretical reference value used was $2 / (n(n-1))$. An important distinction is that this classic bound concerns preserving a particular fixed minimum cut, while the measured success rate counts success when the algorithm returns any minimum cut. Therefore, the theoretical value is used as a conservative reference rather than as an exact prediction of the measured quantity.

Results

The measured success probabilities were as follows:

n	Cycle	Complete	Two-cluster	Theoretical bound
4	1.000	0.803	—	0.1667
6	1.000	0.7124	0.5412	0.0667
8	1.000	0.7304	0.2114	0.0357
10	1.000	0.7540	0.2710	0.0222
12	1.000	0.7812	0.3204	0.0152



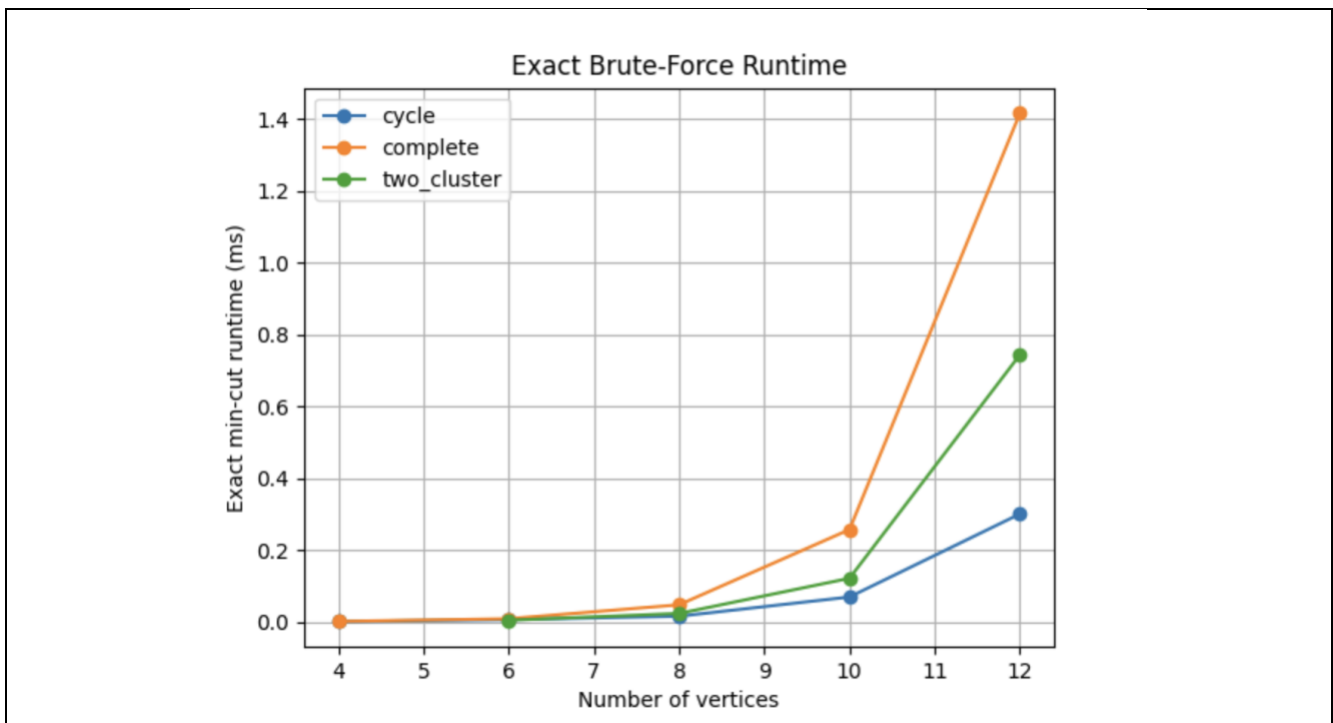
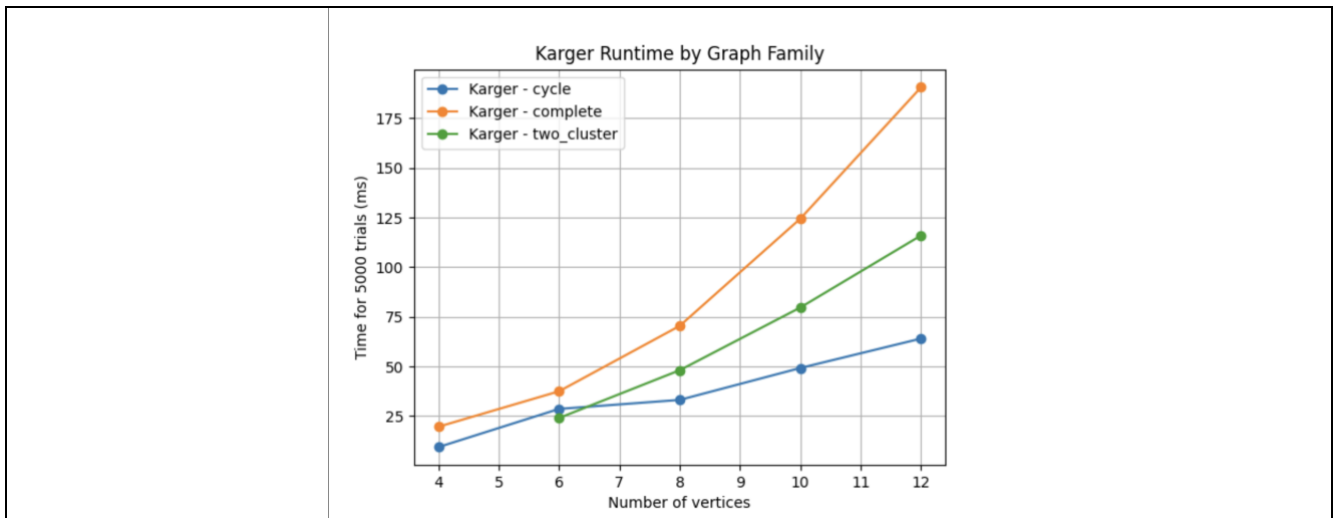
Cycle graphs achieved a measured success rate of 1.0 for every tested size. This is explained by their structure: every cut produced by the contraction process has size two, which is also the minimum cut. Complete graphs also performed much better than the theoretical bound, with success rates of roughly 0.71 to 0.80. The two-cluster graphs were more difficult; for $n = 8$ the measured success rate was approximately 0.2114.

The two-cluster result is intuitive because the graph has a small bottleneck consisting of two bridge edges. If one of those edges is contracted early, that bottleneck cut can no longer be returned. The experiment therefore shows that graph structure has a substantial effect on the probability of success.

Runtime

The total runtime for 5,000 Karger trials increased with graph size and density. The complete graph family provides a clear example:

n	Edges	Karger: 5000 trials (ms)	Exact baseline (ms)
4	6	19.66	0.0010
6	15	37.52	0.0085
8	28	70.47	0.0473
10	45	124.49	0.2569
12	66	190.67	1.4161



These timing columns do not represent identical amounts of work: the Karger measurement contains 5,000 randomised runs, while the exact measurement contains one brute-force execution. They should therefore not be compared as a direct speed ratio. However, their growth patterns are still informative.

At $n = 12$, 5,000 Karger runs took approximately 64 ms for the cycle, 116 ms for the two-cluster graph, and 191 ms for the complete graph. This is consistent with the implementation, because every contraction step scans the edge list and denser graphs require more edge checks. The exact baseline also grows rapidly because it enumerates exponentially many possible partitions.

Interpretation

The central result is that the theoretical lower bound was very conservative for all tested graph families. At $n = 12$, the reference value is approximately 0.0152, whereas the observed success rates ranged from 0.3204 for the two-cluster graph to 1.0 for the cycle graph.

This does not contradict Karger's analysis. The theoretical result is a guarantee rather than a prediction of typical behaviour on every graph. In addition, the experiment counts finding any minimum cut as a success, while the standard proof tracks the survival of a particular fixed minimum cut. The experiments therefore illustrate the difference between a worst-case theoretical guarantee and empirical behaviour on specific graph structures.

Limitations

The largest graph tested had only 12 vertices because the brute-force exact baseline grows exponentially. A larger study could replace this baseline with a more efficient deterministic minimum-cut algorithm. The tested graph families are also highly structured; random graphs and additional families would provide broader coverage. Finally, runtime results depend on implementation choices. This version rebuilds the list of surviving edges during every contraction step, so an optimised implementation may show different timing behaviour.

(c) What I learned

One of the most important things I learned was the different treatment required for parallel edges and self-loops. Parallel edges must remain because each copy changes the probability of selecting that connection. In contrast, self-loops created by contractions must be excluded, because choosing one would not merge two different supernodes and would change the sampling distribution required by Karger's algorithm.

I also gained a better understanding of why a theoretical lower bound should not be interpreted as the probability that will normally be observed. I expected empirical success to be above the bound, but I did not initially expect the cycle graphs to succeed on every run. That result made me examine the structure of the graph rather than immediately assuming the implementation was incorrect.

Another lesson was how strongly graph structure affects randomised algorithms. Cycle, complete, and two-cluster graphs of similar sizes produced substantially different success rates. This showed me that the number of vertices alone does not capture all of the practical behaviour of the algorithm.

Using DSU also clarified how contractions can be represented without continually rebuilding the vertex set. The key invariant is that every original vertex belongs to exactly one current DSU component, and an original edge survives exactly when its endpoints are in different components.

During development, I encountered several practical mistakes, including inconsistencies between header declarations and function definitions, accidentally placing one function inside another, and initially creating a new random-number generator inside every Karger call. Fixing these issues made the program structure clearer and made the experiments reproducible.

(d) AI Use

I used ChatGPT during the project as a development and debugging assistant. I used it to break the task into stages, discuss suitable data representations, plan the Track A empirical study, and troubleshoot C++ and development-environment errors.

Examples of AI-assisted work included suggesting an edge-list plus DSU representation, identifying missing header declarations, diagnosing a function that had accidentally been placed inside another function, helping restructure the random-number generator so it could be reused across trials, and suggesting graph families and an exact brute-force baseline for the experiments.

I did not treat AI-generated suggestions as automatically correct. I compiled and ran each change, checked known minimum cuts on small graphs, compared randomised results against the exact implementation, and corrected issues when compiler output or test results showed that something was wrong.

I also used ChatGPT to help structure the report and identify patterns worth discussing in the experimental results. The numerical values and plots in this report came from my own program and the generated results.csv file.